

Informe — La baja del port del kernel a Go

Versión: 1.0 **Fecha:** 2026-08-10 **Autor:** Claude (por mandato de César Obach) **Objeto:** justificar y documentar la baja de `TODO.md` de la partida «Port del kernel @vergis/policy a Go», y dejar registrado qué la reactivaría.

Resumen ejecutivo

Lo que se dio de baja es una línea de `TODO.md`, no el port. El port sigue siendo una opción técnicamente sana y sigue decidido —diferido— en `docs/adr-001-lenguaje-y-supply-chain.md` §Decisión-2. Lo que se retiró es un **duplicado** de esa decisión que vivía en el libro de pendientes y que había envejecido peor que su fuente.

Tres hechos sostienen la baja:

1. **El TODO era redundante.** Decía lo mismo que el ADR-001, con menos matiz.
2. **El duplicado se volvió falso.** ADR-002 (agosto) reencuadró el driver más fuerte que el ADR-001 citaba —«Custos como producto standalone»— y la línea de `TODO.md` no se enteró.
3. **Ningún disparador vivo tiene demanda hoy.** Quedan `embedding` y librería `WASM`; ninguno tiene un solicitante concreto.

Lo que NO dice este informe: que portar a Go sea mala idea. En el eje de `supply chain`, Go es genuinamente superior, y el ADR-001 ya lo reconoce.

¿Qué es exactamente el port que se evaluaba?

`packages/policy` es el **compilador de policies**: toma la declaración `audience.rls` de un spec y emite el `enforcement` nativo del motor de base de datos (`ROW POLICY` de ClickHouse, `SECURITY POLICY` de Fabric). Es el kernel de seguridad del producto en su parte de autorización de filas.

Medición del terreno al 2026-08-10:

Métrica	Valor medido
Tamaño	1.276 LOC en 10 archivos
Dependencias externas	cero (solo <code>@vergis/botler</code> , interna)
I/O y concurrencia	ninguna — transformación pura de estructuras de datos
Consumidores en el árbol	<code>serve-rls</code> , <code>discovery</code> , <code>engines/fabric</code> , <code>engines/clickhouse</code> , <code>capabilities</code> , <code>miranda</code>
Oráculo de equivalencia	2 property tests diferenciales, 800 iteraciones cada uno (<code>tests/policy.test.ts:184 y :430</code>)

Ese perfil —pequeño, puro, sin dependencias, con oráculo— es justamente lo que haría el port **verificable mecánicamente**: se corre la implementación en Go contra la de TypeScript con los mismos casos aleatorios y se exige igualdad de filas. Esa propiedad **no se ha deteriorado**.

¿Por qué se dio de baja entonces?

Razón 1 — la decisión ya tenía casa, y era otra

El ADR-001 §Decisión·2 dice, textual: «*Se porta —Go es el candidato, por su modelo de supply chain— cuando exista un driver de negocio concreto: Custos como producto standalone, embedding en otro runtime, o distribución como librería/WASM. No antes.*»

Eso es una decisión arquitectónica con su condición de disparo. La línea de `T0D0.md` la repetía en una frase. **Un pendiente que solo repite una decisión ya registrada no es deuda: es ruido con apariencia de trabajo** — cada vez que se abre el archivo, mira feo y no pide nada.

Razón 2 — el duplicado se desincronizó y quedó desinformando

`docs/adr-002-open-core.md` catalogó `packages/policy` (Custos, kernel RLS) como pieza **abierta, prioridad 1**, y situó la frontera comercial en otro sitio: control plane de flota, HA/K8s.

La consecuencia es directa y el `T0D0.md` no la recogía: «**Custos como producto standalone**» **deja de ser un driver de ingreso**. Si Custos standalone llega a existir, nace abierto —lo manda ADR-002—, de modo que ya no puede justificar la inversión de un port por la vía «esto se vende aparte». El disparador más fuerte de los tres del ADR-001 quedó, si no muerto, sí severamente reencuadrado.

Ésta es la mecánica de daño que la Norma 6 nombra: la línea de `T0D0.md` seguía citando una justificación que un documento posterior había invalidado, y lo hacía con la autoridad de un pendiente vigente.

Razón 3 — los disparadores que quedan no tienen demanda

Disparador del ADR-001	Estado al 2026-08-10
Custos como producto standalone comercial	Reencuadrado por ADR-002 — si nace, nace abierto
Embedding en otro runtime	Vivo, sin demanda — no hay solicitante
Distribución como librería / WASM	Vivo, sin demanda — no hay solicitante

Construir contra un disparador sin solicitante es exactamente lo que el ADR-001 prohibió con su «No antes».

Contra-consideración: el port podría encarecerse

Registrada para que la decisión futura la tenga a la vista.

El frente **09 de #113** (`execute-sql-local`, «Motor L») propone que `applyPolicy` —el oráculo del kernel— sea **el motor de filtrado** de la ejecución SQL local, con el argumento de evitar un tercer codegen y su drift. Si eso se construye, el kernel deja de ser una función pura consumida en la frontera y pasa a ser una pieza de ejecución entrelazada con el runtime JS, lo que **sube el costo del port**.

Etiqueta de fiabilidad (Norma 6): esto está **leído del diseño** `work/004-cluster-disenos-backlog-2026-08-07/09-...`, **no medido**, y ese frente **no está construido**. Es una consideración de diseño, no un hecho del árbol.

Implicación práctica: si alguna vez el port se reactiva, conviene decidirlo **antes** de construir el Motor L, no después.

Corrección colateral del ADR-001

Al medir el terreno apareció un defecto en la evidencia del propio ADR-001.

El documento afirmaba «**2.100 iteraciones de property testing diferencial**» como sustento de que el kernel está garantizado por diseño y no por el type system. Lo medido:

- `tests/policy.test.ts:184` — 800 iteraciones (codegen $\equiv$ evaluador de referencia)

- tests/policy.test.ts:430 — 800 iteraciones (Fabric $\equiv$ referencia $\equiv$ ClickHouse)

Total: **1.600 iteraciones**. Contando aserciones diferenciales (el segundo test compara tres vías) serían **2.400**. **Por ningún conteo da 2.100**.

La conclusión del ADR no se cae: el oráculo existe, es diferencial, cubre los dos motores y es suficiente para hacer verificable un port. Lo que falla es la cifra, y la cifra era carga — sostenía el argumento de «garantía por diseño». Corregida en el ADR con nota de corrección visible.

Qué se hizo, concretamente

Archivo	Cambio
TODO.md	La partida se marca dada de baja, con el porqué y el puntero a este informe y al ADR
docs/adr-001-lenguaje-y-supply-chain.md	§Decisión·2 gana un Delta 2026-08-10 con el reencuadre de ADR-002, los disparadores vivos y la contra-consideración del Motor L
docs/adr-001-lenguaje-y-supply-chain.md	La cifra 2.100 se corrige a 1.600 con su desglose y nota de corrección

¿Qué reactivaría el port?

Cualquiera de estas tres, y ninguna se da hoy:

1. **Un solicitante de embedding** — alguien que necesite el kernel dentro de un runtime que no es Node (un agente, un sidecar, un motor de terceros).
2. **Demanda de librería/WASM** — distribuir el compilador de policies como artefacto consumible fuera de Vergis.
3. **Un giro de ADR-002** — que Custos vuelva al lado comercial. Eso exige, por el propio ADR-002, «*derogar este ADR con un acto documentado*».

Qué re-verificar el día que se dispare, antes de escribir la primera línea de Go:

- ¿Sigue packages/policy en cero dependencias externas y en el orden de 1.000–1.500 LOC?
- ¿Sigue el oráculo diferencial verde y cubriendo los motores vigentes?
- ¿Se construyó el Motor L? Si sí, medir cuánto del kernel quedó entrelazado con el runtime JS.
- ¿El puerto AuthorizationProvider sigue siendo la única superficie por la que Vergis consume el kernel? Es la costura que hace el reemplazo barato.

Lectura de una línea

El port a Go **no se descartó: se devolvió a su única casa**, que es el ADR-001 — y de paso esa casa quedó al día con lo que ADR-002 le había cambiado por debajo y con la cifra que citaba mal.

- Generado con Wingworking